

二次予選 解説

2022年2月5日・情報オリンピック日本委員会

第21回日本情報オリンピック 二次予選

問題 1 図書館 2 (Library 2)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

解説担当：戸高空

この問題はスタックと呼ばれるデータ構造で実現できる処理の 1 つである。スタックは C++ の標準ライブラリに含まれているが、これを用いなくても簡単に実装することができる。以下にこの問題における実装の一例を示す。

積まれている本を下から順に十分な長さの配列 `A[1]`, `A[2]`, `A[3]`, ... を用いて記録する。また現在積まれている本の個数を変数 `size` に記録する。新しく本が積まれる場合は `size` に +1 して `A[size]` に本の名前を記録する。本を返却する場合は `A[size]` を出力して `size` を -1 する。

原文：<https://www2.ioi-jp.org/joi/2021/2022-yo2/2022-yo2-t1-review.html>

問題 2 カーペット (Carpet)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

解説担当：平木康傑

小課題については省略し、満点を得られる解法について解説する。

この問題のように探索が求められる問題では、「幅優先探索」と「深さ優先探索」がしばしば有効である。特に、今回は最短の手数を求める必要がある状況なので、幅優先探索を用いるのが妥当である。なお、深さ優先探索を用いると小課題 1, 2, 3 は通るが、満点がとれるほど高速ではない。

また、2 次元グリッドをグラフとみなして、最短経路問題に落とし込むことも考えられる。この場合は幅優先探索のほかに、Dijkstra 法を適用しても解ける。

C++ の解答例は幅優先探索による方針を採っている。 `std::queue` を用いて実装しているが、ほかにも様々な方法がある。

Python の解答例は Dijkstra 法による方針を採っている。3-16 行目ではグリッドをグラフに落とし込んでおり、17-24 行目では Dijkstra 法を実行している。

原文：<https://www2.ioi-jp.org/joi/2021/2022-yo2/2022-yo2-t2-review.html>

問題 3 国土分割 (Land Division)

配点 100点 時間制限 1 sec メモリ制限 1024 MB

解説

解説担当：星井智仁

以下、JOI 国の南端，東端にも境界線を引くものとして考える．

西から 1 本目の南北方向と並行な境界線を西から i マス目の東側に引き，北から 1 本目の東西方向と並行な境界線を北から j マス目の南側に引くことを考える．すると，残りの境界線の引き方は一意に定まる，もしくは達成不可能である．

従って，各 (i, j) の組について $(i, j) = (H, W)$ の場合を除いて，達成可能かを判定すれば良い．

(i, j) の組が $HW - 1$ 通り，各判定は二次元累積和などを用いれば $O(HW)$ で可能であるため，計算量 $O(H^2W^2)$ でこの問題を解くことができる．

原文：<https://www2.ioi-jp.org/joi/2021/2022-yo2/2022-yo2-t3-review.html>

問題 4 飴 2 (Candies 2)

配点 100点 時間制限 2 sec メモリ制限 1024 MB

解説

解説担当：大佐 健人

小課題 1

N が小さいので、食べる飴の集合 2^N 通りを全探索し、条件を満たす食べ方の中から美味しさの和の最大値を求めればよい。これは2進数または再帰関数で実装することができる。

小課題 2

飴 1 から順に飴を食べるかどうかを決める動的計画法を行うことを考える。直前 K 個のうち食べる飴の集合が分かっているならば、次の飴を食べてもよいか分かる。そのため、

$dp[i][S]$: 飴 1 から飴 i までの食べ方を決め、直前の K 個のうち食べる飴の集合が S の時の美味しさの合計の最大値

という DP を行うことで、 $O(NK2^K)$ で解くことができる。

小課題 3

小課題 2 の DP について、直前 K 個のうち 3 個以上食べる状態は無視してよい。そのため、直前に食べた 2 個のみを保存する DP を考える。

$dp[i][j]$: 最後に食べたのが飴 i 、最後から 2 番目に食べたのが飴 j の時の、美味しさの合計の最大値

という DP を行い、すべての (i, j) についての $dp[i][j]$ の最大値を答えとすればよい。ここで、制約から、常に 2 個以上食べる解が最適になることに注意する。

遷移は、 $dp[i][j] \leftarrow A_{\{i\}} + A_{\{j\}}$ で初期化したうえで、

$dp[i][j] \leftarrow \max\{dp[j][k] \mid k \leq \min(j-1, i-K)\} + A_{\{i\}}$

となる。この DP は $O(N^3)$ で動くので、この小課題を解くことができる。

小課題 4

小課題 3 の DP を高速化することで解ける。 $M[i][j] = \max\{dp[i][k] \mid k \leq j\}$ という配列を作り、DP しながらこの配列も更新することで、DP の遷移が

$dp[i][j] \leftarrow M[j][\min(j-1, i-K)] + A_{\{i\}}$

と、 $O(1)$ でできるようになる。これで全体計算量が $O(N^2)$ となったため、この問題の満点が取れる。

原文：<https://www2.ioi-jp.org/joi/2021/2022-yo2/2022-yo2-t4-review.html>

問題 5 交易計画 (Trade Plan)

配点 100点 時間制限 4 sec メモリ制限 1024 MB

解説

解説担当：山縣 龍人 (tatyam)

小課題 1

$Q = 1$ の場合を考えよう。この場合、両端が交易に協力してくれる都市であるような道路のみを取り出してグラフを作り、そのグラフ上で DFS や UnionFind を用いて都市 A から都市 B まで移動できるか判定することで、時間計算量 $O(N + M)$ で解くことができる。

また、 Q 回の交易それぞれで上を行うことで、時間計算量 $O(Q(N + M))$ でこの問題を解くことができる。小課題 1 では、 $N \leq 1000$ 、 $M \leq 1000$ 、 $Q \leq 1000$ と制約が小さいので、このアルゴリズムは十分高速である。

小課題 2

小課題 2 では、すべての州は連結である。(飛び地のない地図を考えてみると良いだろう。) したがって、各交易では、都市 A の属する州と都市 B の属する州がとなり合っているか (あるいは同じであるか) を判定すれば良い。

州 $S[U]$ に属する都市と州 $S[V]$ に属する都市を結ぶ道路があれば、州 $S[U]$ と州 $S[V]$ はとなり合っているという情報を平衡二分探索木 (C++ における `std::set`) などに記録し、各交易で都市 A の属する州と都市 B の属する州がとなり合っているか時間計算量 $O(\log M)$ で判定すれば、時間計算量 $O(N + (M + Q) \log M)$ でこの問題を解くことができる。シラバス外ではあるが、平衡二分探索木の代わりにハッシュテーブルを使うのも良いだろう。

小課題 3

小課題 3 は、小課題 1 の時間計算量 $O(Q(N + M))$ から何か改善したアルゴリズムで解くことを想定している。ここでは、時間計算量 $O(N + M Q^{1/2} \log M + Q(\log M + \log Q))$ のアルゴリズムを解説する。他の方法でも正解できるかもしれない。

小課題 1 の UnionFind による解法を改善することを考える。まず、長さ N の UnionFind を Q 回作ることができないので、UnionFind を、経路圧縮せず変更履歴を持つ undo 可能 UnionFind に変更する。アルゴリズムは以下ようになる。

1. 長さ N の undo 可能 UnionFind を作る。
2. 道路を両端の州によって分類する。
3. 都市 A から都市 B に特産品を届けられるか判定する。
 1. 両端が州 $S[A]$ に属する道路を UnionFind に追加する。
 2. 両端が州 $S[B]$ に属する道路を UnionFind に追加する。
 3. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する道路を UnionFind に追加する。

4. 都市 A から都市 B に移動可能か判定する.
5. 道路の追加を全て undo する.

このままでは、同じ道路が何度も追加・削除されてしまう。そこで、一端の州が同じ交易をまとめて処理する。

1. 長さ N の undo 可能 UnionFind を作る.
2. 道路を両端の州によって分類する.
3. 交易を両端の州によって分類する.
4. 一端が州 $S[A]$ の都市である交易を判定する.
 1. 両端が州 $S[A]$ に属する道路を UnionFind に追加する.
 2. もう一端が州 $S[B]$ の都市である交易を判定する.
 1. 両端が州 $S[B]$ に属する道路を UnionFind に追加する.
 2. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する道路を UnionFind に追加する.
 3. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する交易全てについて、都市 A から都市 B まで移動可能か判定する.
 4. 1. 2. を undo する.
 3. 1. を undo する.

それでもまだ、「両端が州 $S[B]$ に属する道路を UnionFind に追加する」部分で同じ道路が何度も追加・削除されてしまう。この量を抑えるために、 $S[A]$ と $S[B]$ に (両端が州 $S[A]$ に属する道路の数) $>$ (両端が州 $S[B]$ に属する道路の数) または (両端が州 $S[A]$ に属する道路の数) $=$ (両端が州 $S[B]$ に属する道路の数) かつ $S[A] \geq S[B]$ という条件を付け、両端がその州に属する道路が多い州ほどまとめて処理されやすくする。

1. 長さ N の undo 可能 UnionFind を作る.
2. 道路を両端の州によって分類する.
3. 交易を両端の州によって分類する.
 1. (両端が州 $S[A]$ に属する道路の数) $<$ (両端が州 $S[B]$ に属する道路の数) または (両端が州 $S[A]$ に属する道路の数) $=$ (両端が州 $S[B]$ に属する道路の数) かつ $S[A] < S[B]$ であれば、 A と B を交換する.
4. 一端が州 $S[A]$ の都市である交易を判定する.
 1. 両端が州 $S[A]$ に属する道路を UnionFind に追加する.
 2. もう一端が州 $S[B]$ の都市である交易を判定する.
 1. 両端が州 $S[B]$ に属する道路を UnionFind に追加する.
 2. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する道路を UnionFind に追加する.
 3. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する交易全てについて、都市 A から都市 B まで移動可能か判定する.
 4. 1. 2. を undo する.
 3. 1. を undo する.

実は、これで時間計算量 $O(N + M Q^{\{1/2\}} \log M + Q(\log M + \log Q))$ を達成している。これを確かめよう。

「両端が州 $S[B]$ に属する道路を UnionFind に追加する」以外の場所では、各道路は高々 1 回しか UnionFind に追加されないの、この操作がない場合の計算量は $O(N + M \log M + Q(\log M + \log Q))$ である。したがって、「両端が州 $S[B]$ に属する道路を UnionFind に追加する」操作が何回行われるかを調べれば良い。両端がその

州に属する道路の数が $M/Q^{1/2}$ 以上である州を大きな州、 $M/Q^{1/2}$ 未満である州を小さな州と呼ぶことにする。州 $S[B]$ が小さい州である場合、操作は交易 1 回あたり高々 $M/Q^{1/2}$ 回なので、これを Q 回やっても高々 $M Q^{1/2}$ 回である。州 $S[B]$ が大きい州である場合、州 $S[A]$ も大きい州でなければならない。道路が M 個しかないことから、大きい州は高々 $M/(M/Q^{1/2}) = Q^{1/2}$ 個しか存在しないので、両端が州 $S[B]$ に属する道路が追加される回数は高々 $Q^{1/2}$ 回で、全ての道路で合計しても高々 $M Q^{1/2}$ 回である。

「両端が州 $S[B]$ に属する道路を UnionFind に追加する」操作が高々 $2 M Q^{1/2}$ 回しか行われなことが分かったので、時間計算量は $O(N + M Q^{1/2} \log M + Q(\log M + \log Q))$ である。

小課題 4

実は、小課題 2 がこの問題を解くヒントになっている。小課題 2 では、各州が連結であるため、州がとなり合っているかを判定したのであった。これは、1 つの州を 1 つの頂点に縮約して、一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する交易が来たら、一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する道路を追加して判定するものを見ることができる。ここで重要なのは、 $S[A]$ でも $S[B]$ でもない州の縮約が答えに影響を与えないことである。(そもそもそのような州の都市を通ることができないため)

これより、あらかじめ両端が同じ州に属する道路を全て追加してしまえば良いことがわかる。その後は小課題 3 と同様にすると、アルゴリズムは以下のようになる。

1. 長さ N の undo 可能 UnionFind を作る。
2. 道路を両端の州によって分類する。
3. 両端が同じ州に属する道路を UnionFind に追加する。
4. 交易を両端の州によって分類する。
5. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する交易を判定する。
 1. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する道路を UnionFind に追加する。
 2. 一端が州 $S[A]$ に、もう一端が州 $S[B]$ に属する交易全てについて、都市 A から都市 B まで移動可能か判定する。
 3. 1. を undo する。

これで時間計算量は $O(N + M \log M + Q(\log M + \log Q))$ となり、満点を取ることができる。また、「両端が同じ州に属する道路を UnionFind に追加」した後の状態を覚えておくことで、経路圧縮のある通常の UnionFind でも undo することができ、さらに (シラバス外ではあるが) 道路と交易の分類にハッシュテーブルを使うことで、時間計算量が $O(N + (M + Q) \alpha(M))$ となる。

原文：<https://www2.ioi-jp.org/joi/2021/2022-yo2/2022-yo2-t5-review.html>

出典：情報オリンピック日本委員会「第21回日本情報オリンピック JOI 2021/2022 二次予選競技課題」。原資料は CC BY-SA 4.0 で提供されています。

本PDFは各解説ページを問題順に再構成したものです。

<https://www2.ioi-jp.org/joi/2021/2022-yo2/index.html>